PROSIT 2

Groupe :

CARVAL Mathéo - Gestionnaire du temps

LUU Philippe - Animateur

BOUHAMED Allan - Scribe

CHALIES Maeva - Secrétaire

Sommaire :

Sommaire :.....2

1. Contexte.....3
2. Mots inconnus / Notions à maîtriser.....3
3. Problématique.....4
4. Plan d'action (Estimé à 11.5h pour la phase d'étude).....4
5. Réalisation.....5

1. Contexte

L'architecture de l'application d'entreposage est désormais validée. L'analyse des sources a montré qu'elles sont hétérogènes : structurées, semi-structurées et non structurées. Archie avait prévu un référentiel intermédiaire, le Data Lake, qui stocke toutes les données quel que soit leur schéma tout en garantissant leur confidentialité. Il faut maintenant construire l'entrepôt de données qui s'appuie sur ce référentiel, à partir du modèle dimensionnel préparé, et y intégrer les données. Plusieurs difficultés apparaissent : la région n'existe pas dans les données d'entrée, certaines informations utiles sont réparties dans des sources différentes et le format de date de Hive ne correspond pas à celui des fichiers. Ces transformations sont le rôle de l'ETL, mais sa position dans le schéma d'Archie fait douter l'équipe : faut-il transformer avant ou après le chargement ? Le planning laisse peu de marge pour produire ce premier livrable.

2. Mots inconnus / Notions à maîtriser

- **Data Lake** : Référentiel de stockage centralisé qui conserve les données brutes dans leur format d'origine, quel que soit leur schéma.
- **Données structurées / semi-structurées / non structurées** : Tables avec schéma fixe (SQL, CSV) / données à balises ou clés variables (JSON, XML, XLSX hétérogènes) / données sans schéma (PDF, images, texte libre).
- **ETL vs ELT** : Extract-Transform-Load transforme les données avant de les charger ; Extract-Load-Transform les charge brutes puis les transforme dans la plateforme cible.
- **Talend** : Outil ETL graphique : un job enchaîne des composants (lecture, tMap pour joindre et transformer, écriture).
- **Qualification d'un jeu de données** : Évaluation de sa qualité : complétude, unicité, validité, cohérence, exactitude et fraîcheur.
- **Anonymisation / Pseudonymisation** : Suppression irréversible du lien avec la personne / remplacement de l'identifiant par un code (réversible avec une clé, reste soumis au RGPD).
- **Table de faits / Dimension** : Table des mesures à analyser (une ligne par événement) / table des axes d'analyse (temps, lieu, patient…).
- **Clé de substitution** : Identifiant technique propre à l'entrepôt, indépendant de la clé métier de la source.
- **Jointure SQL** : Association de lignes de plusieurs tables sur une colonne commune (INNER JOIN, LEFT JOIN).
- **Hive / HiveQL** : Entrepôt de données de l'écosystème Hadoop, interrogé en SQL ; il attend les dates au format yyyy-MM-dd.
- **Référentiel géographique** : Table de correspondance commune (code INSEE) → département → région.

3. Problématique

Comment intégrer et transformer des données hétérogènes et sensibles issues du Data Lake afin d'alimenter un modèle dimensionnel fiable et conforme au RGPD, capable de répondre aux analyses demandées, et faut-il pour cela transformer les données avant (ETL) ou après (ELT) leur chargement ?

4. Plan d'action (11,5h)

1. Qualification des jeux de données sources : structure, volumétrie et qualité (3h).
2. Organisation du Data Lake et choix entre ETL et ELT (2h).
3. Conception des transformations : anonymisation RGPD, formats de date, enrichissement géographique, croisement de sources (3h).
4. Alimentation du Data Mart avec l'ETL et intégration dans la base NoSQL (Hive) (2h).
5. Traduction des demandes métier en requêtes SQL multi-tables et validation des résultats (1.5h).

5. Réalisation

- **5.1 Qualification des jeux de données (3h)** : Chaque source a été profilée (schéma, volume, valeurs manquantes, doublons, formats) avant toute transformation. Les problèmes relevés guident directement les règles de l'ETL :

Source	Type	Format	Volume	Problèmes de qualité relevés
Base PostgreSQL DATA2023	Structurée	Dump SQL, 13 tables	3,7 M lignes	Dates en texte M/d/yyyy ; 2,35 % de consultations avant la naissance du patient ; 2 tables de test (AAAA, date)
Hospitalisations.csv	Structurée	CSV	2 479 séjours	Dates dd/MM/yyyy ; codes diagnostic absents de la base
Établissements (S2)	Structurée	CSV **;**	3,3 M lignes	Encodages mixtes (UTF-8 / ISO-8859-1) ; 33 % sans code commune ; FINESS sans zéro initial ; annuaire tronqué à 1 048 575 lignes
Satisfaction e-Satis	Semi-structurée	CSV ou XLSX selon l'année	5 335 lignes	Colonnes différentes chaque année ; Windows-1252 ; nombres au format « 75,09 »
Décès INSEE	Structurée, volumineuse	CSV 1,9 Go	25 M lignes	Dates partielles (jour ou mois à 00) ; 169 671 doublons ; codes 99xxx = étranger
Modalités de calcul	Non structurée	PDF	—	Archivé tel quel dans le Data Lake

  Stratégie d'analyse : profiler → écrire une règle par problème (*ex: Des consultations sont datées avant la naissance du patient on fait une règle : date_consultation >= date_naissance et on met dans silver_rejets_consultation et on stoppe le chargement si silver_rejets_consultation > 5%)*

  → isoler les lignes invalides dans une table de rejets (au lieu de les supprimer en silence) → produire un rapport qualité à chaque chargement (Great Expectations) avec des contrôles bloquants.
- **5.2 Organisation du Data Lake et choix ETL / ELT (2h)** : Le Data Lake est découpé en zones successives (architecture médaillon), chacune dans un bucket MinIO :

Zone	Contenu	Règle	Accès
Landing	Fichiers déposés à l'identique (FTP)	Archive, expiration 365 jours	Data engineers
Bronze	Copie brute de chaque source + date et lot d'ingestion	Ajout seul, rejouable	Data engineers
Silver	Une table par entité, typée, dédoublonnée, pseudonymisée + tables de rejets	Aucune donnée directement identifiante	Data engineers
Gold	Modèle dimensionnel (Data Mart)	Seule zone exposée	Application (via Trino)

Critère	ETL (transformer puis charger)	ELT (charger puis transformer)
Où se fait la transformation	Serveur ETL dédié (Talend)	Dans la plateforme cible (Spark, Hive)
Données brutes conservées	Non	Oui, on peut rejouer
Volumes	Limité par le serveur ETL	Profite de la puissance du cluster
Schéma	Défini à l'écriture (schema-on-write)	Défini à la lecture (schema-on-read)
Adapté à	Entrepôt classique, données propres	Data Lake, Big Data, sources variées

Conclusion : la position de l'ETL dans le schéma d'Archie n'est pas une erreur, les deux se défendent. Dans un Data Lake, l'ELT est le plus naturel : on charge brut en Bronze, puis on transforme dans Spark entre Silver et Gold. C'est le choix retenu, car il garde une trace de la donnée d'origine.

- **5.3 Conception des transformations (3h)** : Chaque difficulté du sujet correspond à une transformation précise :

Difficulté	Transformation	Exemple
Région absente	Jointure avec un référentiel INSEE : commune → département (2 caractères, 2A/2B en Corse, 3 en outre-mer) → région (découpage 2016)	75056 → 75 → 11 Île-de-France
Informations dans des sources différentes	Clé de rapprochement normalisée entre sources	Satisfaction ↔ établissement : `identifiant_organisation = 'F' || finess_geo`
Établissement absent des consultations	Rattachement principal du professionnel (activité salariée, puis libérale)	36 % sans rattachement → membre « Non rattaché »
Format de date	Conversion explicite vers yyyy-MM-dd (format Hive) et clé AAAAMMJJ	`to_date(Date, 'M/d/yyyy')` : 3/7/2019 → 2019-03-07
Nombres français	Remplacement de la virgule décimale	« 75,09 » → 75.09
Doublons	Conservation de l'enregistrement le plus récent sur la clé métier	169 671 décès en double écartés
Valeurs nulles	Obligatoire nul → rejet ; descriptif nul → « Non renseigné »	Clé -1 dans les dimensions

Anonymisation RGPD appliquée dès le passage en Silver :

```
-- pseudonymisation : même patient = même code, mais non réversible sans le sel
sha2(concat('<sel secret>', Id_patient), 256)   AS id_patient_pseudo,
year(to_date(Date, 'M/d/yyyy'))                  AS annee_naissance,   -- généralisation
substr(Code_postal, 1, 2)                        AS departement
-- Nom, Prenom, Adresse, Email, Tel, Num_Secu : non repris (minimisation)
```

Il s'agit techniquement d'une pseudonymisation. Elle devient proche d'une anonymisation pour l'utilisateur final grâce à la généralisation (tranches d'âge) et au masquage des effectifs inférieurs à 10.

- **5.4 Alimentation du Data Mart et intégration NoSQL (2h)** : Les dimensions sont chargées en premier, puis les faits qui récupèrent leurs clés de substitution par jointure (lookup). Les tables sont stockées en colonnes (Parquet / Delta) et déclarées

dans le catalogue Hive Metastore, ce qui les rend interrogeables en HiveQL ou via Trino. L'équivalent Talend de chaque étape :

Étape	Composant Talend	Équivalent dans notre projet
Lire PostgreSQL	tPostgresqlInput	Lecture JDBC Spark, 8 partitions en parallèle
Lire un CSV	tFileInputDelimited	`spark.read.csv` avec encodage explicite
Joindre, transformer	tMap	`join` + `withColumn` en PySpark
Dédoublonner	tUniqRow	`dropDuplicates` / MERGE sur la clé métier
Écrire dans Hive	tHiveOutput	Écriture Delta + déclaration dans le Hive Metastore
Planifier	Scheduler Talend	DAG Airflow

```
CREATE EXTERNAL TABLE gold.fait_hospitalisation (
  num_hospitalisation STRING, id_date_entree INT, sk_patient BIGINT,
  sk_etablissement BIGINT, sk_diagnostic BIGINT, duree_sejour_jours INT,
tranche_age STRING
) PARTITIONED BY (annee INT) STORED AS PARQUET
LOCATION 's3a://gold/fait_hospitalisation';
```

Résultat du chargement : 6 dimensions (1,67 M lignes) et 4 faits (25,95 M lignes), **0 orphelin** sur les 13 clés étrangères contrôlées. Le premier passage a même été bloqué à juste titre : des décès antérieurs à 1970 n'existaient pas dans le calendrier, qui a été étendu à 1880.

- **5.5 Traduction des demandes en requêtes SQL (1h30)** : Méthode : repérer la mesure (ce qu'on compte), les axes (par quoi), le filtre (quand, où), puis choisir la table de faits et les dimensions à joindre. Exemple « taux de consultation par diagnostic en 2020 » :

```
SELECT d.code_diag, d.libelle, COUNT(*) AS nb,
       CAST(COUNT(*) AS DOUBLE) / SUM(COUNT(*)) OVER () AS taux
FROM gold.fait_consultation f
JOIN gold.dim_diagnostic d ON f.sk_diagnostic = d.sk_diagnostic
WHERE f.annee = 2020
GROUP BY d.code_diag, d.libelle
ORDER BY nb DESC LIMIT 20;
```

Exemple multi-tables « nombre de décès par région en 2019 » :

```
SELECT l.libelle_region, COUNT(*) AS nb_deces
FROM gold.fait_deces f
JOIN gold.dim_localisation l ON f.sk_localisation = l.sk_localisation
JOIN gold.dim_temps t        ON f.id_date = t.id_date
WHERE t.annee = 2019
GROUP BY l.libelle_region ORDER BY nb_deces DESC;
```

Validation : chaque total est comparé au comptage direct dans la source (par exemple le nombre de séjours de l'export d'hospitalisations), et les définitions des taux (numérateur / dénominateur) sont documentées pour que tout le monde calcule la même chose.